

From Vibe Coding to Spec-Driven Agentic Engineering

A Hands-On Implementation using Claude Code

PART IV

Instructor: Muhammad Arif Butt, PhD

About The Instructor



Dr. Muhammad Arif Butt
Asst. Prof. Department of Data Science
University of Punjab, Lahore
arif@pucit.edu.pk
https://arif.excaliat@gmail.com
https://arifbutt.me
https://www.linkedin.com/in/dr-arif-butt/
https://youtube.com/learnwitharif
https://github.com/arifpucit
https://huggingface.co/arif-butt

- **Education:**
  - Graduation from Pakistan Military Academy, Kakul
  - MPhil CS from University of Punjab, Lahore
  - PhD CS from University of Punjab, Lahore
- **Experience:**
  - Served in field/staff/instructional posts in Pakistan Army
  - Assistant Professor, Department of Data Science
  - Founder Excaliat (https://excaliat.com/en)
  - Founder FalconHunt (https://falconhunt.org/)
  - Co-Founder Tbox Solutionz (https://tboxsolutionz.com/)
- **Research Interest:**
  - Embedded and Real-Time Operating Systems
  - Vulnerability Analysis, Binary Exploitation & Exploit Development
  - AI-Driven Cybersecurity and Securing AI systems

Scheme of Sessions

Part - I: Overview of Agentic Coding Assistants

Part – II: Claude Ecosystem, Installing & using Claude Code

Part – III: Slash Commands, Context Window & `.claude` Directory

Part – IV: Workflow of Building a Basic Vulnerable Web Application

Part – V: Exploiting Vulnerable Web Application

Part – VI: Workflow of Implementing Features and Fixing Bugs

Part – VII: Skill Marketplace and Writing your Own Skills

Part – VIII: Creating and using Subagents & Agent Teams

Part – IX: Model Context Protocol

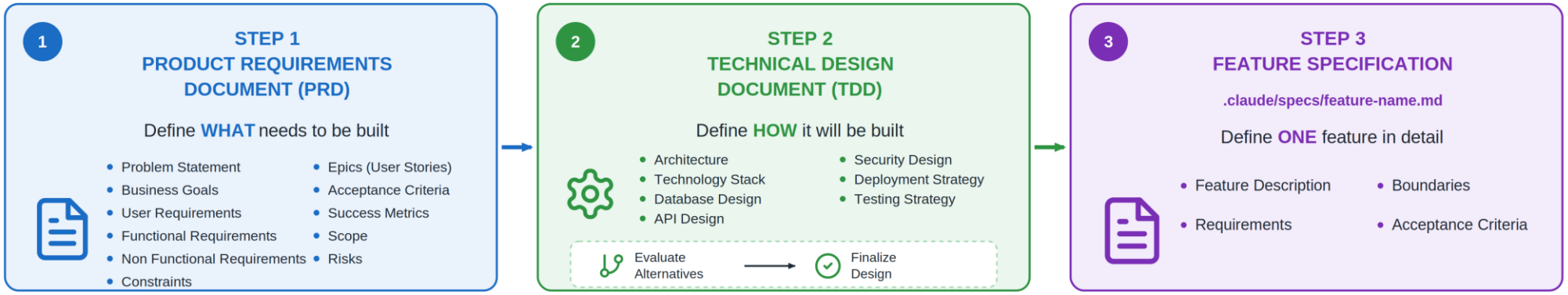
Part – X: Creating and using Hooks

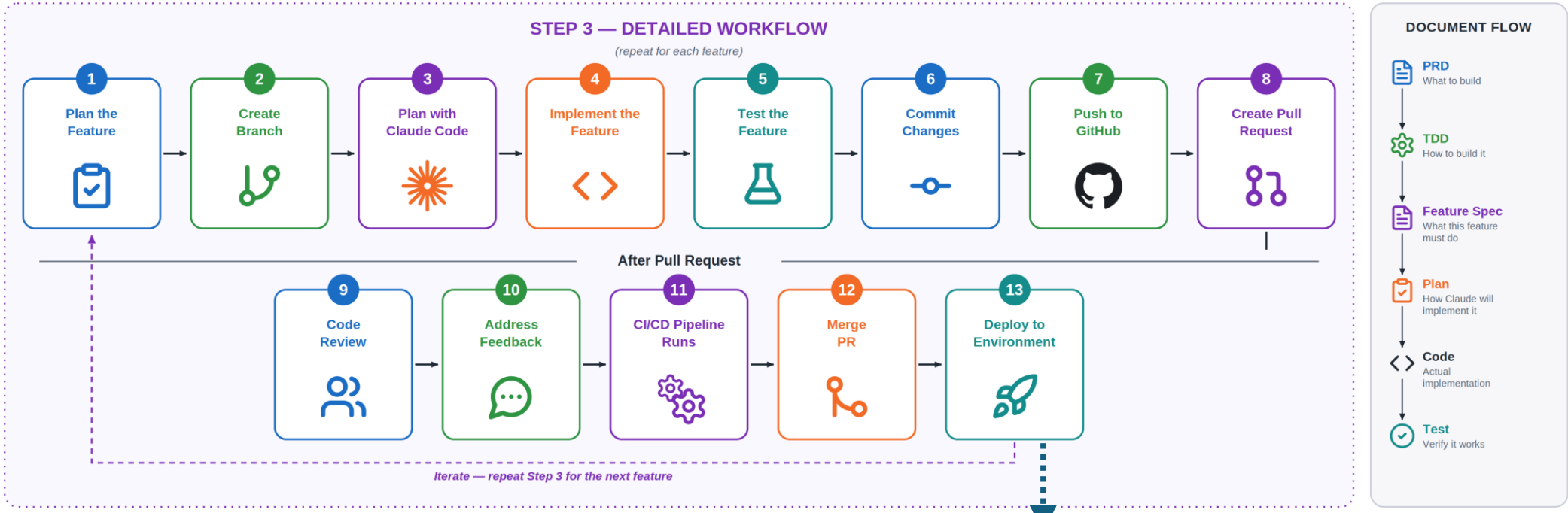
Part – XI: Creating and using Plugins

Part 4
Workflow of Building a Basic Vulnerable Web App

Instructor: Muhammad Arif Butt, PhD

Spec-Driven Development with Claude Code







https://arifpucit.github.io/cc-workflow/

PRD (Product Requirements Document)

Purpose: Defines what the product is, why it exists, and who it serves.

- Educational Security Lab
- Problem Statement
- Proposed Solution
- Target Audience
- Core Features
- 8 Intentional Vulnerabilities
- Non-Functional Requirements
- Learning Outcomes
- Success Metrics

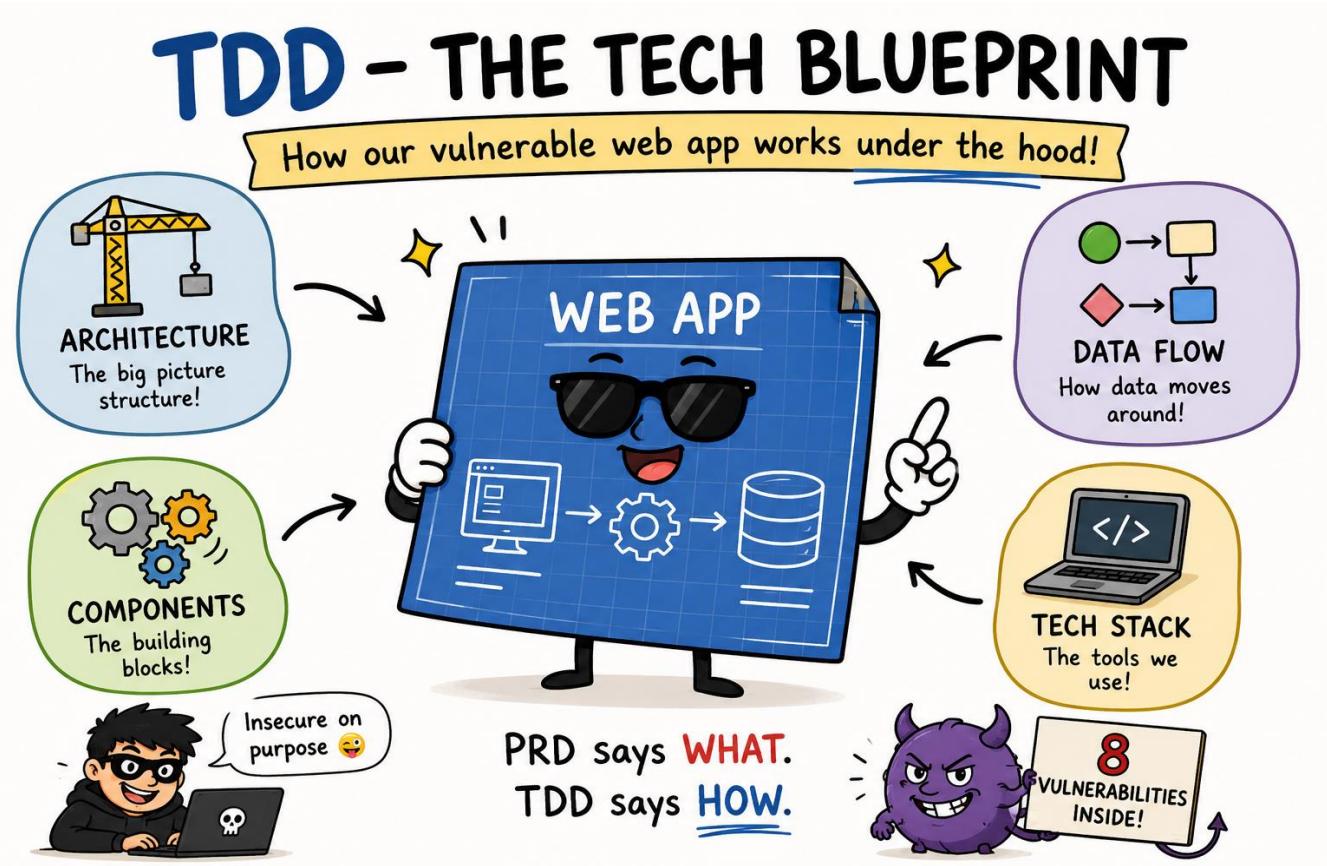


https://github.com/arifpucit/vuln-web-app/blob/main/docs/PRD.md

TDD (Technical Design Document)

Purpose: Explains architecture, technologies, components, data flow, and technical implementation.

- System Architecture
- Technology Stack
- Backend Components
- Frontend Components
- Database Design
- Application Data Flow
- Security Vulnerability Design
- Technical Requirements
- Testing & Verification

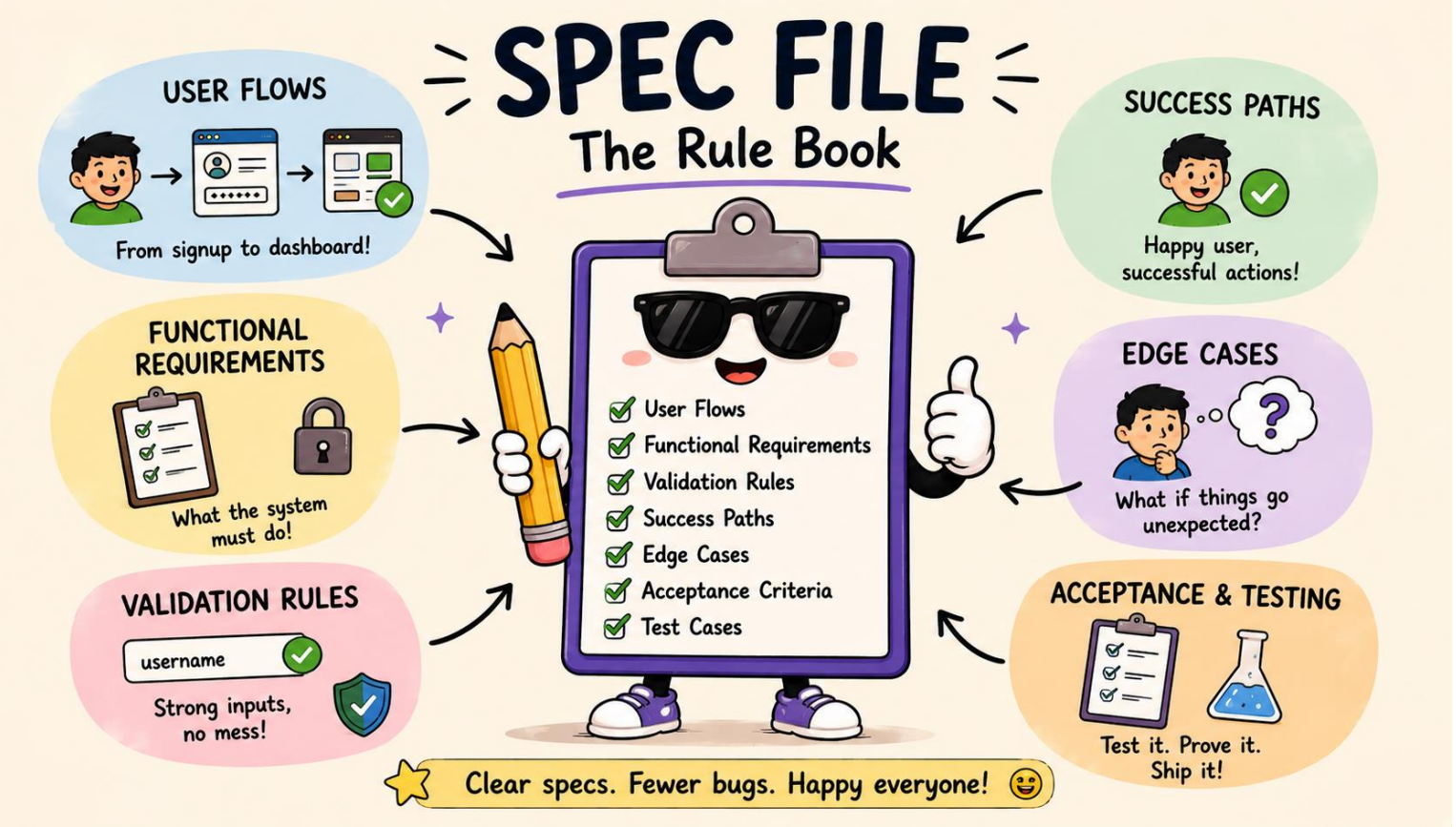


https://github.com/arifpucit/vuln-web-app/blob/main/docs/TDD.md

Spec File

Purpose: Defines exact system behavior, user flows, validation rules, and UI requirements.

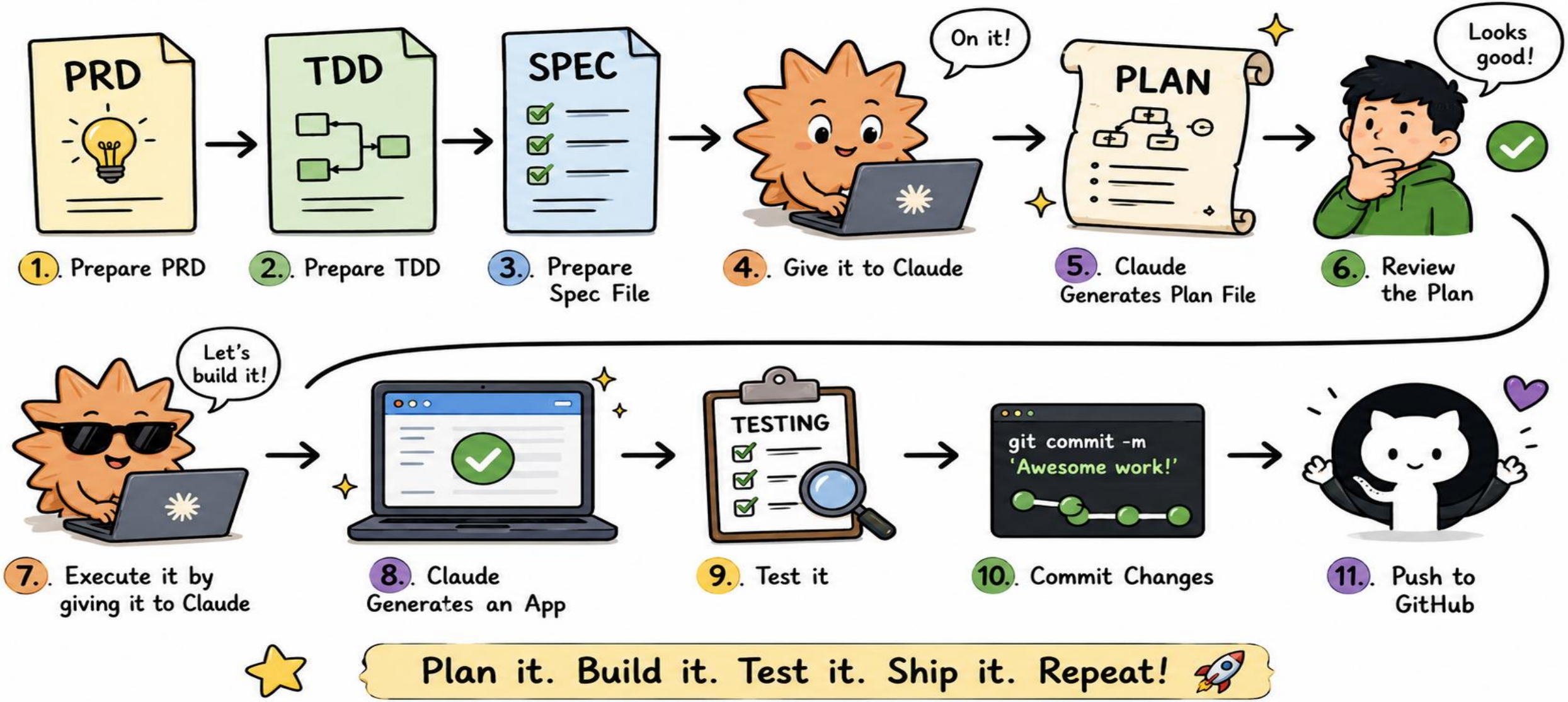
I. Application Runtime Behavior
II. User Flows
III. Functional Requirements
IV. UI Specifications
V. Session State & Data Lifecycle
VI. Success Paths & Alternate Paths
VII. Edge Cases
VIII. Acceptance Criteria & Testing



https://github.com/arifpucit/vuln-web-app/blob/main/.claude/specs/app-foundation.md

https://github.com/arifpucit/vuln-web-app/blob/main/.claude/specs/app-foundation-plan.md

Documents to Production: Workflow



Programming Languages & Tech Stacks

Frontend Development
Runs in the browser — the structure, style and interactivity users see.

</> HTML
USED FOR
Defines the structure & content of every web page.

FRAMEWORKS / TOOLS
Templating: EJS, Pug, Handlebars · pairs with CSS + JS

CSS
USED FOR
Styles & lays out pages — colours, spacing, responsive design.

FRAMEWORKS / TOOLS
Tailwind CSS, Bootstrap, Sass, Bulma

JS JavaScript
USED FOR
Adds interactivity in the browser; also runs servers via Node.js.

FRAMEWORKS / TOOLS
Front: React, Vue, Angular, Svelte · Back: Node.js, Express

How the layers connect
FRONTEND (browser) — HTML · CSS · JavaScript
sends a request ↓
BACKEND (server) — Python · Java · Go · PHP · C# · Rust
queries ↓
DATABASE (stored data) — SQL · NoSQL · Vector

Backend Development
Runs on the server — business logic, APIs, authentication and data access.

Py Python
USED FOR
Backend, automation, data science & AI / ML. Beginner-friendly.

FRAMEWORKS / TOOLS
Django, Flask, FastAPI · AI: PyTorch, TensorFlow

J Java
USED FOR
Large-scale enterprise backends and native Android apps.

FRAMEWORKS / TOOLS
Spring Boot, Hibernate, Jakarta EE · Android SDK

PHP PHP
USED FOR
Server-side web development; powers a huge share of the web.

FRAMEWORKS / TOOLS
Laravel, Symfony, CodeIgniter · WordPress

Go Go
USED FOR
Fast, concurrent backend & cloud services, APIs, microservices.

FRAMEWORKS / TOOLS
Gin, Echo, Fiber · gRPC, Docker / Kubernetes tooling

.N .NET (C#)
USED FOR
Backend, web, desktop, mobile & games on Microsoft's platform.

FRAMEWORKS / TOOLS
ASP.NET Core, Blazor · MAUI · Unity (games)

Systems & Mobile
Close to the hardware — operating systems, performance-critical software and devices.

C C
USED FOR
Operating systems, embedded systems & drivers — fast and low-level.

FRAMEWORKS / TOOLS
Standard library, POSIX · SDL, GTK

C++ C++
USED FOR
High-performance systems, game engines & desktop software.

FRAMEWORKS / TOOLS
Qt (desktop), Unreal Engine (games), Boost

Rs Rust
USED FOR
Memory-safe systems & backend services, no garbage collector.

FRAMEWORKS / TOOLS
Actix, Axum, Rocket (web) · Tokio (async)

Obj Objective-C
USED FOR
Legacy iOS & macOS apps (today the standard is Swift / SwiftUI).

FRAMEWORKS / TOOLS
Cocoa, Cocoa Touch · Apple SDKs

Sw Swift
USED FOR
Apple's modern language for iOS, macOS, iPadOS & watchOS apps.

FRAMEWORKS / TOOLS
SwiftUI, UIKit · Vapor (server-side)

Databases
Where the data lives — chosen by the shape of the data and how it will be queried.

SQL Relational
PostgreSQL · MySQL · MariaDB · SQLite · SQL Server · Oracle

Best for: structured data, complex queries and reliable transactions (ACID) with clear relationships between tables.

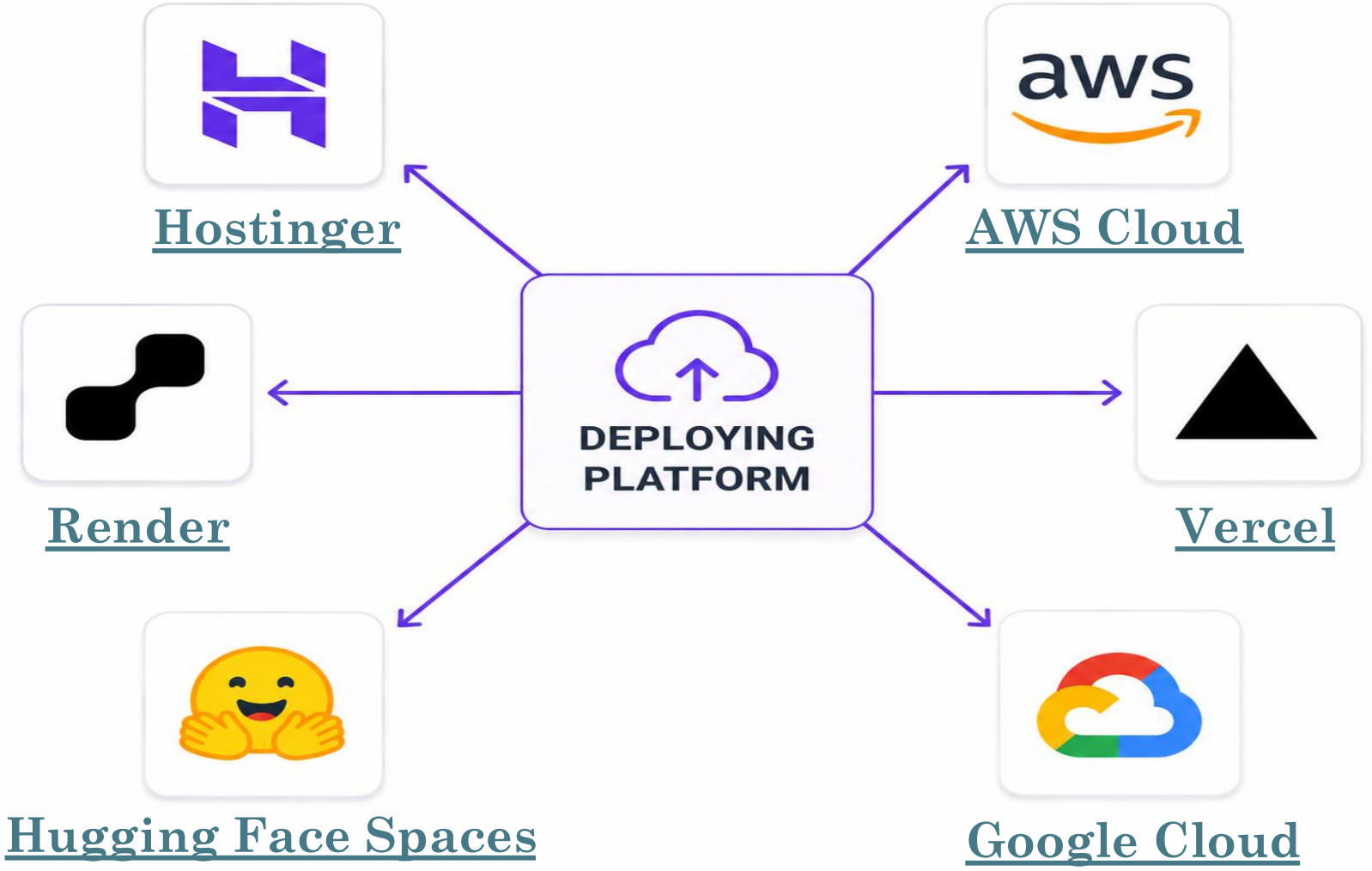
NoSQL Non-relational
MongoDB (doc) · Redis (key-value) · Cassandra (wide-col) · DynamoDB · Neo4j (graph) · Firebase

Best for: flexible / changing schemas, massive scale, caching and real-time apps with varied data shapes.

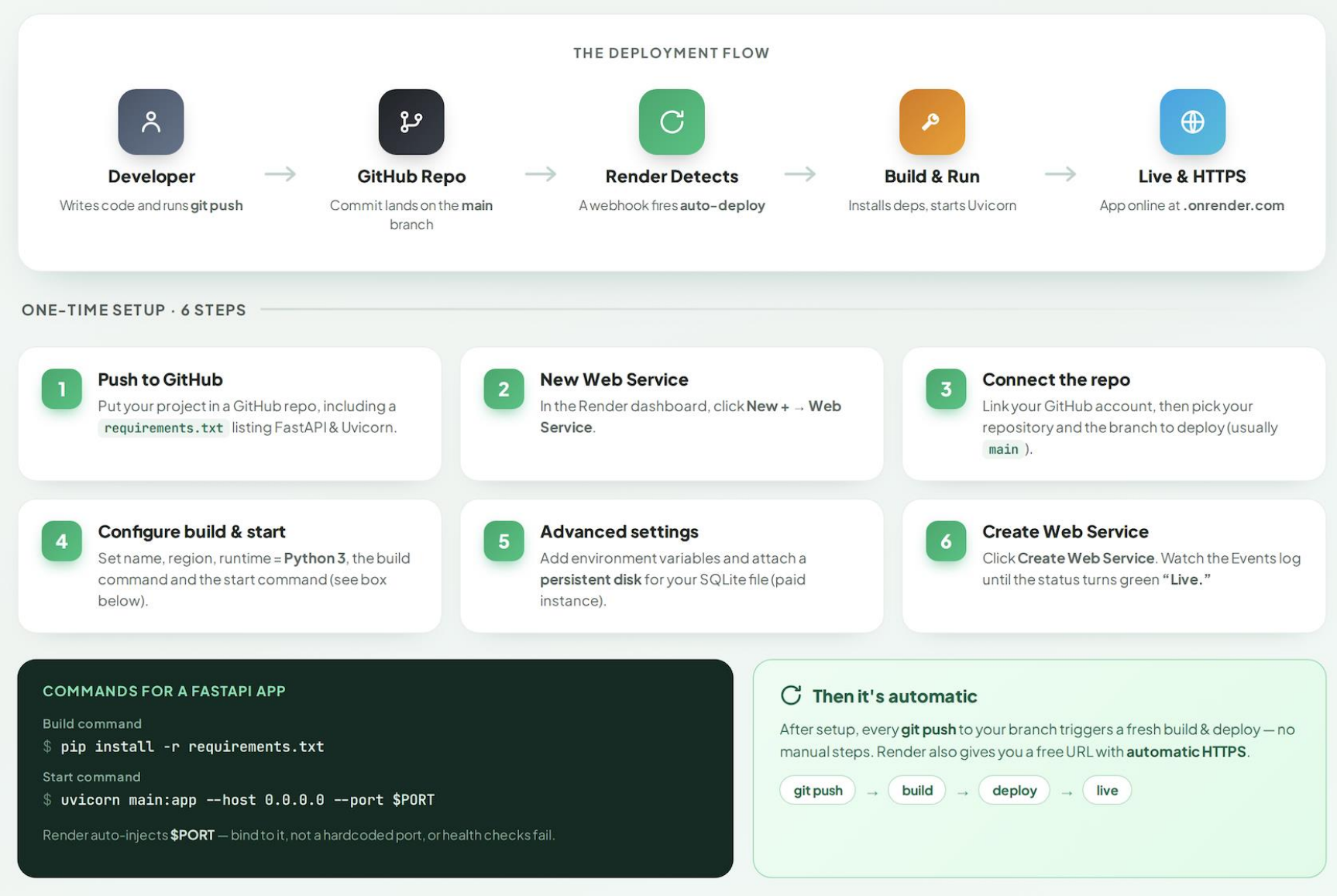
Vector AI / Embeddings
Pinecone · Weaviate · Milvus · Qdrant · Chroma · pgvector

Best for: similarity search over embeddings — powers semantic search, recommendations and RAG for LLM / AI apps.

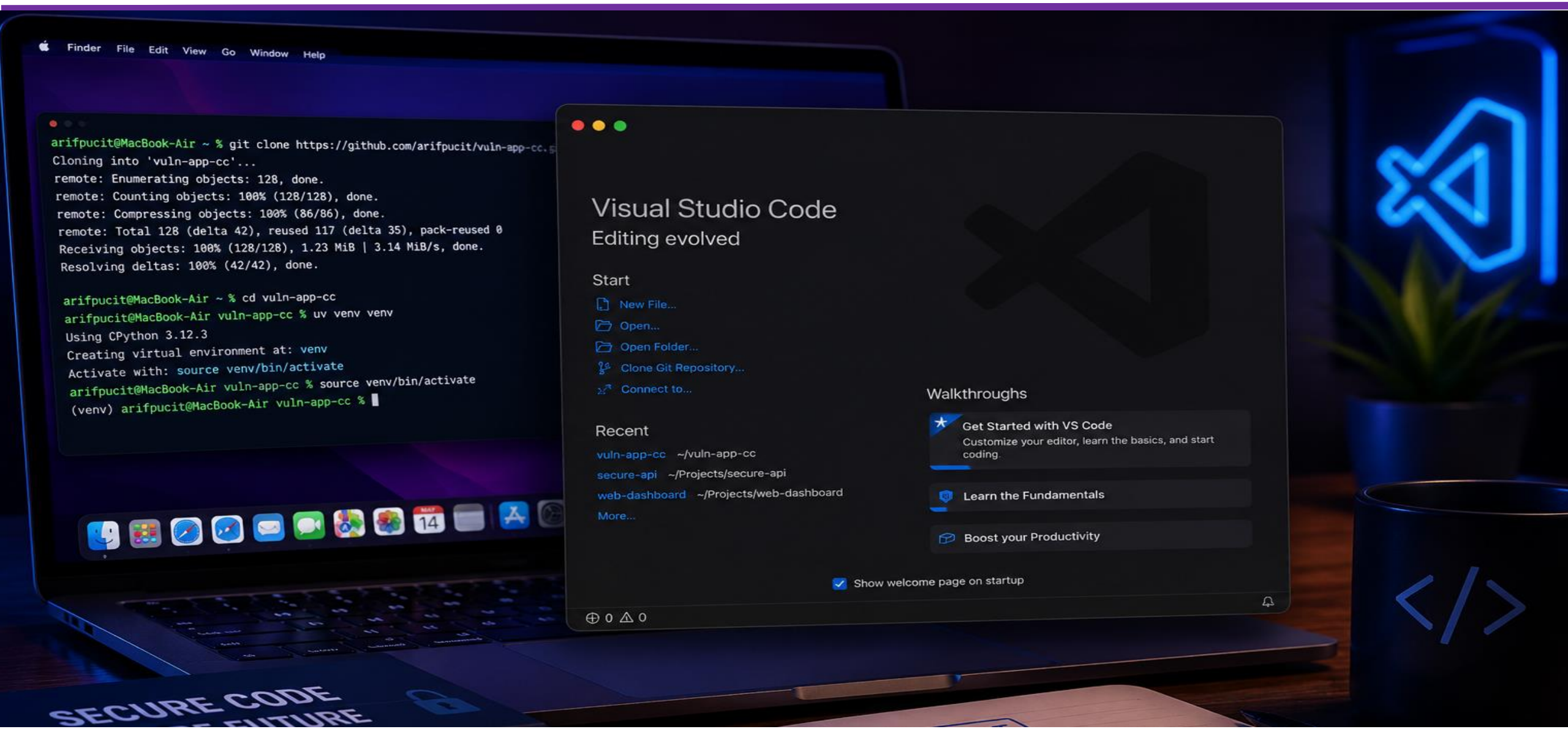
Deployment Platforms



Deployment: GitHub -> Render



Hands on Practice



Q/A



Thank you!